

รายงานสัปดาห์ที่ 8 (Basic)

Automating with CSV and JSON Data

สรุปเนื้อหา บันทึกผลแล็บ ผลการรันจริง และตอบคำถามสำคัญประจำสัปดาห์

โปรเจกต์: data-processor (src/, data/, main.py)

1. ผลลัพธ์การเรียนรู้ (Learning Outcomes)

- เข้าใจโครงสร้างและกรณีการใช้งานของไฟล์ CSV และ JSON
- ใช้โมดูล csv ของ Python อ่านและเขียนข้อมูล CSV ได้อย่างถูกต้อง รวมถึงการจัดการหัวตาราง (headers) และตัวคั่นแบบต่าง ๆ
- ใช้โมดูล json แปลงข้อมูลไป-กลับระหว่าง Python object กับ JSON รวมถึงข้อมูลที่ซ้อนชั้น (nested)
- เขียนสคริปต์ประมวลผลข้อมูล เช่น อ่าน แปลง กรอง และเขียนข้อมูลที่มีโครงสร้าง
- แก้ปัญหาที่เกี่ยวข้องกับรูปแบบข้อมูล การเข้ารหัส (encoding) และการ parse ไฟล์ CSV/JSON ได้

2. สรุปเนื้อหาบทเรียน

2.1 ข้อมูลแบบมีโครงสร้าง (Structured Data Files)

ไฟล์แบบ plaintext อย่าง CSV และ JSON ต่างจากไฟล์แบบ binary ตรงที่เปิดอ่านและตรวจสอบด้วยตาเปล่าได้ทันที และใช้งานร่วมกันได้แทบทุกระบบ จึงนิยมใช้ในงานบันทึก log, ไฟล์ตั้งค่า (configuration), การรับ-ส่งข้อมูลผ่าน API, การเก็บข้อมูลอย่างง่าย และการนำเข้า-ส่งออกข้อมูลระหว่างโปรแกรม

2.2 การทำงานกับไฟล์ CSV (โมดูล csv)

- พื้นฐานรูปแบบ CSV: ตัวคั่น (comma, tab, semicolon), กฎการใส่เครื่องหมายคำพูด (quoting) และการจบบรรทัด
- csv.reader() — อ่านทีละแถว คืนค่าเป็น list ของ string เหมาะกับข้อมูลเรียบง่าย
- csv.DictReader() — อ่านแต่ละแถวเป็น dictionary โดยใช้แถวแรกเป็นหัวตาราง เหมาะกับข้อมูลที่มีโครงสร้างชัดเจน
- csv.writer() — เขียน list ของ list/tuple ลงไฟล์ CSV
- csv.DictWriter() — เขียน dictionary ลงไฟล์ CSV โดยจับคู่ key กับ fieldnames
- ปรับตัวคั่นด้วย delimiter=, ปรับอักขระคำพูดด้วย quotechar= และควบคุมการใส่เครื่องหมายคำพูดด้วย quoting=csv.QUOTE_MINIMAL/ALL/NONNUMERIC
- ข้อควรระวังเรื่องชนิดข้อมูล: ข้อมูลที่อ่านจาก CSV เป็น string ทั้งหมดเสมอ ต้องแปลงชนิดเองด้วย int() หรือ float() ก่อนนำไปคำนวณ

2.3 การทำงานกับไฟล์ JSON (โมดูล json)

JSON เก็บข้อมูลเป็นคู่ key-value, array และอ็อบเจกต์ซ้อนกันได้ ซึ่งแมปตรงกับ dictionary และ list ของ Python จึงทำงานร่วมกันได้อย่างเป็นธรรมชาติ

- Serialization (Python → JSON): `json.dumps()` แปลงเป็น string ส่วน `json.dump()` เขียนลงไฟล์โดยตรง ปรับความอ่านง่ายด้วย `indent` และเรียงคีย์ด้วย `sort_keys`
- Deserialization (JSON → Python): `json.loads()` แปลงจาก string ส่วน `json.load()` อ่านจากไฟล์โดยตรง
- ข้อมูลซ้อนชั้น: เข้าถึงด้วยการไล่ key และ index ต่อกัน เช่น `data[0]['details']['brand']`
- ต่างจาก CSV ตรงที่ JSON เก็บชนิดข้อมูลไว้ในตัว (number, boolean, null) จึงไม่ต้องแปลงชนิดเองเมื่ออ่านกลับมา

2.4 การประยุกต์ใช้และแนวปฏิบัติที่ดี

- การทำความสะอาดข้อมูล (data cleaning): ลบข้อมูลซ้ำ จัดการค่าที่หายไป
- การรวมข้อมูลอย่างง่าย (aggregation): หาผลรวม ค่าเฉลี่ย
- การกรองข้อมูลตามเงื่อนไข (filtering)
- การแปลงข้อมูลไป-กลับระหว่าง CSV และ JSON
- การใช้ JSON เป็นไฟล์ตั้งค่าของแอปพลิเคชัน

3. สรุปผลการทำแล็บ: Data Processor

แล็บนี้สร้างโปรเจกต์ `data-processor` ที่แยกตรรกะ CSV และ JSON ออกเป็นโมดูลตามหลักการออกแบบแบบโมดูล โดยมี `main.py` เป็นตัวสั่งการลำดับงานทั้งหมด

3.1 โครงสร้างโปรเจกต์

ไฟล์	หน้าที่
<code>src/utils.py</code>	ตั้งค่า logging และฟังก์ชันสร้างโฟลเดอร์ถ้ายังไม่มี
<code>src/csv_handler.py</code>	คลาส <code>CSVHandler</code> — อ่าน CSV เป็น dict, เขียน dict ลง CSV, ประมวลผลข้อมูลการขาย
<code>src/json_handler.py</code>	คลาส <code>JSONHandler</code> — อ่าน/เขียน JSON, อัปเดตสต็อก, เพิ่มสินค้าใหม่
<code>main.py</code>	จุดเริ่มต้น สั่งงาน workflow ทั้งฝั่ง CSV และ JSON ตามลำดับ
<code>data/</code>	เก็บไฟล์ดั้งเดิม (input) และไฟล์ผลลัพธ์ (output)

3.2 ส่วนประมวลผล CSV

- อ่าน `input_sales.csv` (ข้อมูลการขาย 8 รายการ) ด้วย `csv.DictReader`
- คำนวณยอดขายรวมและราคาเฉลี่ยต่อรายการ
- กรองเฉพาะรายการที่ Amount ตั้งแต่ 100 ขึ้นไป
- เขียนผลลัพธ์พร้อมแถวสรุป (SUMMARY) ลง `output_filtered_sales.csv` ด้วย `csv.DictWriter`

3.3 ส่วนประมวลผล JSON

- อ่าน `input_inventory.json` (สินค้า 3 รายการ มีข้อมูลซ้อนชั้นใน details)
- อัปเดตสต็อก `PROD001` จาก 50 เป็น 60 และ `PROD003` จาก 80 เป็น 75

- เพิ่มสินค้าใหม่ PROD004 (Gaming Headset) ที่มี list ซ้อนอยู่ในฟิลด์ features
- เขียนข้อมูลที่อัปเดตแล้วลง output_updated_inventory.json แบบจัดรูปแบบให้อ่านง่าย (indent=2)

4. ผลการรันจริงจาก Terminal (Test Results)

รันคำสั่ง `python main.py` จริงบนเครื่อง โปรแกรมทำงานจบครบทุกขั้นตอนโดยไม่มี error (exit code 0) ผลลัพธ์ที่ได้ตรงกับที่คำนวณด้วยมือทุกค่า (ภาพนี้เป็นผลการรันหลังแก้บั๊ก log เข้าแล้ว ดูรายละเอียดการแก้ในหัวข้อ 5.3)

```

week8/basic — PowerShell (กำลังแก้ไข)

PS C:\xampp\htdocs\proj\week8\basic> python main.py
2026-09-08 11:13:11,954 - INFO - Directory ensured: C:\xampp\htdocs\proj\week8\basic\data
2026-09-08 11:13:11,954 - csv_handler - INFO - CSVHandler initialized.
2026-09-08 11:13:11,954 - json_handler - INFO - JSONHandler initialized.
2026-09-08 11:13:11,954 - __main__ - INFO - --- Starting CSV Data Processing Workflow ---
2026-09-08 11:13:11,954 - csv_handler - INFO - Successfully read 8 rows from 'C:\xampp\htdocs\proj\week8\basic\data\input_sales.csv'.
2026-09-08 11:13:11,954 - csv_handler - INFO - Sales data processed. Total Sales: 1902.00, Filtered Count: 3
2026-09-08 11:13:11,955 - __main__ - INFO - Summary of all sales: {'Total Sales': '1902.00', 'Average Item Price': '237.75', 'Filtered Sales Count': 3}
2026-09-08 11:13:11,955 - INFO - Directory ensured: C:\xampp\htdocs\proj\week8\basic\data
2026-09-08 11:13:11,955 - csv_handler - INFO - Successfully wrote 4 rows to 'C:\xampp\htdocs\proj\week8\basic\data\output_filtered_sales.csv'.
2026-09-08 11:13:11,955 - __main__ - INFO - --- Starting JSON Data Processing Workflow ---
2026-09-08 11:13:11,955 - json_handler - INFO - Successfully read JSON from 'C:\xampp\htdocs\proj\week8\basic\data\input_inventory.json'.
2026-09-08 11:13:11,955 - json_handler - INFO - Updated stock for product 'PROD001' to 60.
2026-09-08 11:13:11,955 - json_handler - INFO - Updated stock for product 'PROD003' to 75.
2026-09-08 11:13:11,956 - json_handler - INFO - Added new product: Gaming Headset.
2026-09-08 11:13:11,956 - INFO - Directory ensured: C:\xampp\htdocs\proj\week8\basic\data
2026-09-08 11:13:11,956 - json_handler - INFO - Successfully wrote JSON to 'C:\xampp\htdocs\proj\week8\basic\data\output_updated_inventory.json'.
2026-09-08 11:13:11,956 - __main__ - INFO - --- Data Processing Workflows Completed ---

```

4.1 ผลลัพธ์ฝั่ง CSV — output_filtered_sales.csv

OrderID	Product	Amount	Price	Customer
101	Laptop	1200.50	1200.50	Alice
104	Monitor	300.00	300.00	Charlie
107	Headphones	150.00	150.00	Alice
SUMMARY	Total/Avg	1902.00	237.75	Count: 3

ตรวจสอบความถูกต้อง: ยอดรวมทั้ง 8 รายการ = 1200.50 + 25.00 + 75.25 + 300.00 + 45.75 + 90.00 + 150.00 + 15.50 = 1902.00 และราคาเฉลี่ย = 1902.00 ÷ 8 = 237.75 ตรงกับผลลัพธ์ที่โปรแกรมคำนวณได้ ส่วนรายการที่ผ่านเงื่อนไข Amount ≥ 100 มี 3 รายการ (Laptop, Monitor, Headphones) ถูกต้อง

4.2 ผลลัพธ์ฝั่ง JSON — output_updated_inventory.json

รายการ	ก่อน	หลัง	สถานะ
PROD001 — Laptop Pro (stock)	50	60	อัปเดตสำเร็จ
PROD002 — Mechanical Keyboard (stock)	120	120	ไม่เปลี่ยนแปลง
PROD003 — Ergonomic Mouse (stock)	80	75	อัปเดตสำเร็จ
PROD004 — Gaming Headset	(ไม่มี)	stock 90	เพิ่มใหม่สำเร็จ

5. คำถาม-คำตอบ และประเด็นที่พบระหว่างทำแล็บ

5.1 csv.reader กับ csv.DictReader ต่างกันอย่างไร ควรใช้ตัวไหน?

csv.reader คืนแต่ละแถวเป็น list ต้องอ้างอิงข้อมูลด้วยตำแหน่งคอลัมน์ เช่น row[2] ซึ่งอ่านยากและพียงันที่ถ้าลำดับคอลัมน์ในไฟล์เปลี่ยน ส่วน csv.DictReader คืนแต่ละแถวเป็น dictionary ที่ใช้แถวแรกเป็นหัวตาราง จึงอ้างอิงด้วยชื่อคอลัมน์ได้ เช่น row['Amount'] อ่านเข้าใจง่ายกว่าและทนต่อการสลับลำดับคอลัมน์ ในแล็บนี้จึงเลือกใช้ DictReader เพราะข้อมูลการขายมีหัวตารางชัดเจน

5.2 ทำไมข้อมูลจาก CSV ต้องแปลงชนิดเอง แต่ JSON ไม่ต้อง?

เพราะ CSV เป็นไฟล์ข้อความล้วนที่ไม่มีการเก็บชนิดข้อมูลไว้เลย ทุกค่าที่อ่านออกมาจึงเป็น string เสมอ ถ้านำ '1200.50' ไปบวกกันตรง ๆ จะกลายเป็นการต่อ string ไม่ใช่การบวกเลข จึงต้องแปลงด้วย float() ก่อน ขณะที่ JSON มีชนิดข้อมูลกำกับอยู่ในไวยากรณ์ของตัวเอง (ตัวเลขไม่ใช่เครื่องหมายคำพูด, มี true/false/null) โมดูล json จึงแปลงเป็นชนิดของ Python ให้อัตโนมัติตั้งแต่ตอนอ่าน

5.3 บั๊กที่พบและการแก้ไข: ข้อความ log ถูกพิมพ์ซ้ำ 2 บรรทัด

อาการที่พบ

เมื่อรันโปรแกรมครั้งแรกด้วยโค้ดตามต้นฉบับ พบว่าข้อความ log ทุกบรรทัดถูกพิมพ์ออกมา 2 ครั้ง โดยบรรทัดหนึ่งมีชื่อโมดูลกำกับ (เช่น csv_handler - INFO - ...) และอีกบรรทัดไม่มีชื่อโมดูล ทำให้ output ยาวเกินจำเป็นถึง 31 บรรทัด และอ่านยาก

```
week8/basic - PowerShell

PS C:\xampp\htdocs\proj\week8\basic> python main.py
2026-09-08 11:04:41,248 - INFO - Directory ensured: C:\xampp\htdocs\proj\week8\basic\data
2026-09-08 11:04:41,248 - csv_handler - INFO - CSVHandler initialized.
2026-09-08 11:04:41,248 - INFO - CSVHandler initialized.
2026-09-08 11:04:41,248 - json_handler - INFO - JSONHandler initialized.
2026-09-08 11:04:41,248 - INFO - JSONHandler initialized.
2026-09-08 11:04:41,248 - __main__ - INFO - --- Starting CSV Data Processing Workflow ---
2026-09-08 11:04:41,248 - INFO - --- Starting CSV Data Processing Workflow ---
2026-09-08 11:04:41,249 - csv_handler - INFO - Successfully read 8 rows from 'C:\xampp\htdocs\proj\week8\basic\data\input_sales.csv'.
2026-09-08 11:04:41,249 - INFO - Successfully read 8 rows from 'C:\xampp\htdocs\proj\week8\basic\data\input_sales.csv'.
2026-09-08 11:04:41,249 - csv_handler - INFO - Sales data processed. Total Sales: 1902.00, Filtered Count: 3
2026-09-08 11:04:41,249 - INFO - Sales data processed. Total Sales: 1902.00, Filtered Count: 3
2026-09-08 11:04:41,249 - __main__ - INFO - Summary of all sales: {'Total Sales': '1902.00', 'Average Item Price': '237.75', 'Filtered Sales Count': 3}
2026-09-08 11:04:41,249 - INFO - Summary of all sales: {'Total Sales': '1902.00', 'Average Item Price': '237.75', 'Filtered Sales Count': 3}
2026-09-08 11:04:41,249 - INFO - Directory ensured: C:\xampp\htdocs\proj\week8\basic\data
2026-09-08 11:04:41,249 - csv_handler - INFO - Successfully wrote 4 rows to 'C:\xampp\htdocs\proj\week8\basic\data\output_filtered_sales.csv'.
2026-09-08 11:04:41,249 - INFO - Successfully wrote 4 rows to 'C:\xampp\htdocs\proj\week8\basic\data\output_filtered_sales.csv'.
2026-09-08 11:04:41,249 - __main__ - INFO - --- Starting JSON Data Processing Workflow ---
2026-09-08 11:04:41,249 - INFO - --- Starting JSON Data Processing Workflow ---
2026-09-08 11:04:41,250 - json_handler - INFO - Successfully read JSON from 'C:\xampp\htdocs\proj\week8\basic\data\input_inventory.json'.
2026-09-08 11:04:41,250 - INFO - Successfully read JSON from 'C:\xampp\htdocs\proj\week8\basic\data\input_inventory.json'.
2026-09-08 11:04:41,250 - json_handler - INFO - Updated stock for product 'PROD001' to 60.
2026-09-08 11:04:41,250 - INFO - Updated stock for product 'PROD001' to 60.
2026-09-08 11:04:41,250 - json_handler - INFO - Updated stock for product 'PROD003' to 75.
2026-09-08 11:04:41,250 - INFO - Updated stock for product 'PROD003' to 75.
2026-09-08 11:04:41,250 - json_handler - INFO - Added new product: Gaming Headset.
2026-09-08 11:04:41,250 - INFO - Added new product: Gaming Headset.
2026-09-08 11:04:41,250 - INFO - Directory ensured: C:\xampp\htdocs\proj\week8\basic\data
2026-09-08 11:04:41,250 - json_handler - INFO - Successfully wrote JSON to 'C:\xampp\htdocs\proj\week8\basic\data\output_updated_inventory.json'.
2026-09-08 11:04:41,250 - INFO - Successfully wrote JSON to 'C:\xampp\htdocs\proj\week8\basic\data\output_updated_inventory.json'.
2026-09-08 11:04:41,250 - __main__ - INFO - --- Data Processing Workflows Completed ---
2026-09-08 11:04:41,250 - INFO - --- Data Processing Workflows Completed ---
```

ภาพที่ 1: ผลการรันก่อนแก้ไข — สังเกตว่าทุกข้อความปรากฏซ้ำกัน 2 บรรทัด (รวม 31 บรรทัด)

การวิเคราะห์สาเหตุ

ต้นเหตุอยู่ที่ไฟล์ src/utils.py ซึ่งตั้งค่า logging ซ้อนกัน 2 ชั้น:

- บรรทัดบนสุดของไฟล์เรียก logging.basicConfig(...) ซึ่งติดตั้ง handler ให้กับ root logger ไว้แล้ว 1 ตัว
- ฟังก์ชัน setup_logging() สร้าง logger ประจำโมดูล แล้วเพิ่ม StreamHandler ให้อีก 1 ตัว
- โดยค่าเริ่มต้น logger ลูกจะ ส่งต่อ (propagate) ข้อความขึ้นไปให้ logger แม่เสมอ ดังนั้นข้อความเดียวกัน จึงถูกพิมพ์ทั้งจาก handler ของโมดูลเอง และจาก handler ของ root logger รวมเป็น 2 ครั้ง

วิธีแก้ไขที่เลือกใช้

เพิ่มคำสั่ง `logger.propagate = False` เพียงบรรทัดเดียวใน `setup_logging()` เพื่อสั่งไม่ให้ `logger` ของแต่ละโมดูลส่งข้อความต่อขึ้นไปให้ `root logger` ข้อความจึงถูกพิมพ์จาก `handler` ของโมดูลตัวเองเพียงครั้งเดียว

```
src/utils.py — ฟังก์ชัน setup_logging()

def setup_logging(name, level=logging.INFO):
    """Sets up a logger for specific modules."""
    logger = logging.getLogger(name)
    logger.setLevel(level)
+ # ไม่ส่งข้อความต่อขึ้นไปให้ root logger (ที่ basicConfig ตั้งค่าไว้แล้ว)
+ # มิฉะนั้นข้อความเดียวกันจะถูกพิมพ์ซ้ำ 2 ครั้ง
+ logger.propagate = False
    if not logger.handlers:
        ch = logging.StreamHandler()
        formatter = logging.Formatter('%(asctime)s - %(name)s - ...')
        ch.setFormatter(formatter)
        logger.addHandler(ch)
    return logger
```

ภาพที่ 2: โค้ดที่แก้ไขใน `src/utils.py` — บรรทัดที่ไฮไลต์สีเขียวคือส่วนที่เพิ่มเข้าไป

ผลหลังแก้ไข

หลังแก้ไข ข้อความ `log` ไม่ซ้ำอีกต่อไป จำนวนบรรทัดลดลงจาก 31 บรรทัด เหลือ 17 บรรทัด
อ่านง่ายขึ้นชัดเจน และที่สำคัญคือ ผลลัพธ์การประมวลผลข้อมูลยังคงเหมือนเดิมทุกค่า (ยอดรวม 1902.00, ค่าเฉลี่ย 237.75, กรองได้ 3 รายการ, อัปเดตสต็อกและเพิ่มสินค้าใหม่สำเร็จ)
ยืนยันว่าการแก้ไขนี้กระทบเฉพาะการแสดงผล `log` เท่านั้น ไม่ได้เปลี่ยนตรรกะการทำงานของโปรแกรม (ดูภาพผลการรันหลังแก้ไขได้ในหัวข้อ 4)

หมายเหตุ: บรรทัด `Directory ensured: ...` ที่ไม่มีชื่อโมดูลกำกับนั้นถูกต้องแล้ว เพราะมาจากฟังก์ชัน `ensure_directory_exists()` ที่เรียก `logging.info()` ผ่าน `root logger` โดยตรงตามการออกแบบเดิมของโค้ด ไม่ได้เกิดจากการพิมพ์ซ้ำ

5.4 การแก้ปัญหาที่พบบ่อย (Debugging)

ปัญหา	สาเหตุ	วิธีแก้
<code>FileNotFoundError</code>	ไฟล์ตั้งต้นไม่อยู่ในโฟลเดอร์ <code>data/</code>	ตรวจสอบว่ามี <code>input_sales.csv</code> และ <code>input_inventory.json</code> อยู่จริง
<code>KeyError</code>	ชื่อหัวตารางไม่ตรงกับ <code>key</code> ที่เรียกใช้	ตรวจสอบตัวสะกดและช่องว่างหน้า-หลังชื่อคอลัมน์ให้ตรงกัน
<code>json.JSONDecodeError</code>	ไวยากรณ์ JSON ผิด	ตรวจสอบจุดทศนิยม และเครื่องหมายคำพูด หรือใช้ตัวตรวจสอบ JSON
<code>ValueError: could not convert string to float</code>	คอลัมน์ตัวเลขมีอักขระอื่นปน เช่น <code>\$</code> หรือ <code>,</code>	ทำความสะอาดข้อมูลก่อนแปลงเป็น <code>float()</code> และดักด้วย <code>try-except</code>

ตัวอักษรเพี้ยนในไฟล์ผลลัพธ์	encoding ไม่ตรงกัน	ระบุ encoding='utf-8' ทั้งตอนอ่านและเขียนเสมอ
-----------------------------	--------------------	---

6. บทสรุป

สัปดาห์นี้ได้ฝึกทำงานกับไฟล์ข้อมูลที่มีโครงสร้างจริงทั้งสองรูปแบบที่พบบ่อยที่สุด โดย CSV เหมาะกับข้อมูลตารางแบบที่มีคอลัมน์ชัดเจน ส่วน JSON เหมาะกับข้อมูลที่ซ้อนชั้นและมีชนิดข้อมูลหลากหลาย สิ่งที่ได้เรียนรู้ไม่ใช่แค่การเรียกใช้โมดูล csv และ json แต่รวมถึงการออกแบบโปรแกรมแบบโมดูลที่แยกหน้าที่ชัดเจน การใส่ error handling และ logging เพื่อให้ตรวจสอบการทำงานย้อนหลังได้ และการตรวจสอบผลลัพธ์ที่ได้กับการคำนวณด้วยมือ ซึ่งเป็นนิสัยสำคัญของการทำงานกับข้อมูลจริง

นอกจากนี้ การพบและแก้บั๊ก log ข้างในหัวข้อ 5.3 ยังเป็นบทเรียนสำคัญว่า โปรแกรมที่ "รันผ่าน" ไม่ได้แปลว่าถูกต้องสมบูรณ์เสมอไป การสังเกตความผิดปกติของ output แล้วไล่หาสาเหตุจนเจอต้นตอที่แท้จริง (พฤติกรรม propagate ของ logger) คือทักษะการดีบั๊กที่จำเป็นพอ ๆ กับการเขียนโค้ดให้ทำงานได้

อ้างอิง: เอกสารประกอบการสอน "LabX BASIC: Automating with CSV and JSON Data" และเอกสารโค้ดประกอบแล็บสัปดาห์ที่ 8

หมายเหตุ: ภาพผลการรันสร้างจากข้อความ output จริงที่ได้จากการรันบนเครื่อง (ไม่ใช่ภาพจับหน้าจอ OS โดยตรง) คำทั้งหมดในภาพเป็นผลลัพธ์จริงจากการรันจริง